UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E DA TERRA
DEPARTAMENTO DE COMPUTAÇÃO

Eduardo Henrique do Lago Silva 202611002854
Helena Carvalho Leal 202611000223
Poliana Rafaela Moraes de Lira Lima - Matrícula 202521001446
Rafael Takeguma Goto - Matrícula 202611000170
Renato Fernandes Pereira - Matrícula 202521001606
Thomás Silva de Araujo - Matrícula 202521000574
Wilson Dias Martins Neto - Matrícula 202611009022

Inferência Utilizando LLMs para o Domínio Jurídico
Relatório Grupo 1

Professor(a): Glauco de Figueiredo Carneiro

São Cristóvão/SE

2026

Eduardo Henrique do Lago Silva 202611002854
Helena Carvalho Leal 202611000223
Poliana Rafaela Moraes de Lira Lima - Matrícula 202521001446
Rafael Takeguma Goto - Matrícula 202611000170
Renato Fernandes Pereira - Matrícula 202521001606
Thomás Silva de Araujo - Matrícula 202521000574
Wilson Dias Martins Neto - Matrícula 202611009022

Inferência Utilizando LLMs para o Domínio Jurídico
Relatório Grupo 1

Relatório submetido ao Programa de Pós Graduação do Departamento de Computação da Universidade Federal de Sergipe, como comprovação parcial à aprovação na disciplina de Disciplina de Tópicos Avançados em Engenharia de Software e Sistemas da Informação I.

Professor(a): Glauco de Figueiredo Carneiro

Aracaju/SE
2026

Sumário do Documento

1. Introdução 5

2. Objetivo 5

3. Infraestrutura e Ambiente de Execução 5

4. Engenharia de Dados e Modelagem PostgreSQL 5

5. Metodologia do Juiz (Engenharia de Rubricas) 7

6. Análise de Resultados e Estatística 8

7. Desenvolvimento da Interface de Auditoria (Front-end Vue.js) 11

8. Tutorial de Reprodução (Restore do Ambiente) 13

Apêndices 14

1. Introdução

Para a referente atividade os integrantes da equipe atuaram como curadores utilizando os datasets:

- Dataset J1 (Questões Abertas): maritaca-ai/oab-bench (01-70 de 210);
- Dataset J2 (Múltipla Escolha): eduagarcia/oab_exams (001 A 738 de 2210 questões).

O qual foi trabalhado por divisão em 7 lotes:

- Lote 1. Poliana: Abertas (01 a 10) | Múltipla Escolha (001 a 106);
- Lote 2. Thomas: Abertas (11 a 20) | Múltipla Escolha (107 a 212);
- Lote 3. Renato: Abertas (21 a 30) | Múltipla Escolha (213 a 318);
- Lote 4. Eduardo: Abertas (31 a 40) | Múltipla Escolha (319 a 424);
- Lote 5. Helena: Abertas (41 a 50) | Múltipla Escolha (425 a 530);
- Lote 6. Rafael: Abertas (51 a 60) | Múltipla Escolha (531 a 636);
- Lote 7. Wilson: Abertas (61 a 70) | Múltipla Escolha (637 a 738).

2. Objetivo

O projeto visa superar as limitações de métricas puramente lexicais (ROUGE/BLEU) na avaliação de respostas jurídicas complexas. Implementamos um framework de LLM-as-a-Judge para auditar a acurácia técnica e a segurança normativa de modelos de linguagem aplicados ao contexto da OAB, utilizando uma infraestrutura relacional para garantir rastreabilidade total.

3. Infraestrutura e Ambiente de Execução

Para garantir um ambiente de produção fiel à realidade, utilizamos a seguinte stack:

- **Virtualização:** Hypervisor Proxmox (KVM/QEMU).
- **Servidor:** VM Ubuntu 24.04.4 LTS.
- **Banco de Dados:** PostgreSQL 16.13.
- **Acesso:** Túnel SSH via porta 1989 para comunicação da aplicação.
- **Armazenamento:** Estrutura de diretórios em /opt/judge com permissões específicas para o usuário postgres.

4. Engenharia de Dados e Modelagem PostgreSQL

A persistência foi desenhada para suportar o ciclo de vida completo dos dados.

4.1. Estrutura de Importação (Stored Procedures)

Desenvolvemos três procedimentos armazenados para automatizar a carga de dados massivos, garantindo a integridade dos relacionamentos:

1. importar_perguntas_abertas(): Processa arquivos JSONL do dataset maritaca-ai/oab-bench.
2. importar_perguntas_fechadas(): Processa dados do eduagarcia/oab_exams.
3. importar_respostas(): Vincula as saídas dos modelos candidatos (Llama, Gemini, Qwen) às suas respectivas perguntas.
4. importar_avaliacoes_juiz(): Processa arquivos JSONL das avaliações humanas

4.2. Rastreabilidade

O banco judge_db permite auditar não apenas a nota, mas o Chain-of-Thought (Raciocínio) de cada juiz, vinculando-o aos metadados de temperatura e precisão dos candidatos.

4.3. Modelagem de Dados

A arquitetura do banco de dados judge_db foi projetada para garantir a integridade referencial e a rastreabilidade exigida pela auditoria jurídica . Implementamos identificadores do tipo IDENTITY e colunas específicas para métricas de performance e subcritérios de rubrica.

4.3.1. Tabela modelos

Esta tabela funciona como um catálogo centralizado tanto para os Modelos Candidatos (que geraram as respostas na Atividade 1) quanto para os Modelos Juízes (utilizados na Atividade 2).

- **id_modelo**: Chave primária (PK) do tipo integer, definida como IDENTITY para garantir unicidade autoincrementada.
- **nome_modelo**: Campo do tipo text que armazena a nomenclatura oficial da IA (ex: 'Llama-3.1-8b-instant').
- **versao**: Campo text para controle de versão do modelo (ex: '3.1', '3.3').
- **parametro_precisao**: Campo text destinado ao registro de hiperparâmetros de execução, como temperatura ou quantização (ex: 'temperature:0.1').

4.3.2. Tabela datasets

Responsável por organizar e categorizar os conjuntos de dados que compõem o experimento.

- **id_dataset**: Chave primária (PK) do tipo integer (IDENTITY).
- **nome_dataset**: Campo text que identifica a fonte dos dados, como 'OAB_Exams' ou 'maritaca-ai/oab-bench'.
- **dominio**: Campo text que, para este projeto, é padronizado como 'Jurídico'.

4.3.3. Tabela perguntas

Armazena o repositório original de questões e as respectivas "respostas ouro" (gabarito oficial) utilizadas para o cálculo de similaridade .

- **id_pergunta**: Chave primária (PK) do tipo integer (IDENTITY).
- **id_dataset**: Chave estrangeira (FK) que estabelece o relacionamento de integridade com a tabela de datasets.
- **id_peca**: Campo integer adicional (não previsto no esquema original), utilizado pela equipe para categorizar o tipo de peça processual ou área temática da questão.
- **enunciado**: Campo text que contém a descrição completa da questão jurídica.
- **resposta_ouro**: Campo text contendo a fundamentação ou alternativa correta definida por especialistas humanos.
- **metadados**: Campo text destinado a informações auxiliares, como ano do exame ou nível de dificuldade.

4.3.4. Tabela respostas_atividade_1

Esta tabela registra o histórico de execuções dos modelos candidatos realizadas na etapa anterior, servindo como o objeto de auditoria para o Juiz-IA .

- **id_resposta**: Chave primária (PK) do tipo integer (IDENTITY).
- **id_pergunta**: Chave estrangeira (FK) que vincula a resposta ao enunciado correspondente na tabela perguntas.
- **id_modelo**: Chave estrangeira (FK) que identifica qual IA candidata (ex: Llama-3.1-8B) gerou o texto.
- **texto_resposta**: Campo text contendo o conteúdo integral gerado pelo modelo.
- **tempo_inferencia_ms**: Campo do tipo real que armazena a latência da geração em milissegundos.
- **data_geracao**: Campo timestamp with time zone para registro cronológico da execução.

4.3.5. Tabela avaliacoes_juiz

É o núcleo da Atividade 2, onde os vereditos dos modelos Juízes (Gemini) são consolidados para análise estatística .

- **id_avaliacao**: Chave primária (PK) do tipo integer (IDENTITY).
- **id_resposta_ativa1**: Chave estrangeira (FK) que aponta para a resposta auditada.
- **id_modelo_juiz**: Chave estrangeira (FK) que identifica qual versão do Gemini atuou como o "Ministro-IA".
- **nota_atribuida**: Campo real que armazena a nota final calculada pelo Juiz (escala de 1 a 5).
- **nota_humana**: Campo integer destinado à nota atribuída pelo especialista da equipe, fundamental para o cálculo da Correlação de Spearman.

- **chain_of_thought**: Campo text que persiste o raciocínio explicativo e a auditoria de alucinações.
- **tokens_prompt** e **tokens_completion**: Campos integer para monitoramento de consumo e eficiência de tokens.

4.3.6. Tabela rubrica_subcriterios

Adição exclusiva da Equipe 1, esta tabela permite uma avaliação multidimensional, decompondo a nota final em critérios específicos.

- **id_subcriterio**: Chave primária (PK) do tipo integer (IDENTITY).
- **id_avaliacao**: Chave estrangeira (FK) que vincula o subcritério à sua respectiva avaliação central.
- **criterio**: Campo text que descreve a dimensão avaliada (ex: "Formalidade", "Acurácia Normativa", "Clareza").
- **nota_criterio**: Campo integer (1 a 5) para a pontuação específica daquele critério.
- **justificativa**: Campo text detalhando o porquê da nota naquele subcritério.
- **peso**: Campo real que define a importância relativa do critério na composição da nota final.

5. Metodologia do Juiz (Engenharia de Rubricas)

O protocolo foi desenhado para realizar uma meta-avaliação que prioriza a fidelidade normativa e a detecção de alucinações, garantindo que qualquer invenção de dispositivos legais resulte na nota mínima.

O framework opera sob as seguintes diretrizes:

- **Persona de Alta Autoridade:** O Juiz assume a identidade de um Ministro do Supremo Tribunal Federal e Professor Doutor em Direito, garantindo que a avaliação seja pautada na precisão da linguagem culta e no domínio absoluto das normas brasileiras.
- **Escala Ordinal Rigorosa:** A avaliação utiliza uma escala de 1 a 5, na qual a Nota 1 é obrigatoriamente aplicada a respostas que citam leis inexistentes ou confundem institutos básicos, enquanto a Nota 5 é para respostas que demonstram raciocínio jurídico e citam jurisprudência relevante.
- **Auditoria de Similaridade Semântica:** Além da correção factual, o protocolo exige uma estimativa qualitativa da similaridade textual (inspirada na métrica BERT Score), comparando a "distância" entre a intenção do gabarito humano e a resposta gerada pelo candidato (modelos de IA).
- **Transparência via Chain-of-Thought:** O processo exige a exposição do raciocínio explicativo (*Chain-of-Thought*), permitindo que a equipe audite se o Juiz está sendo rigoroso ou se apresenta falhas de interpretação.

Esta abordagem garante que o sistema não apenas quantifique o acerto, mas qualifique a fundamentação jurídica, separando a fluidez textual da competência técnica normativa.

- **Persona:** Ministro do STF e Professor Doutor em Direito.
- **Modelos Juízes:** Gemini 3 Flash, Gemini 3.1 Flash Lite e Gemini 2.5 Flash.
- **Lógica de Avaliação:** Escala ordinal de 1 a 5 .
    - Nota 1**:** Atribuída automaticamente em caso de Alucinação Normativa (invenção de leis/artigos).
    - Nota 5: Reservada para respostas impecáveis que igualam ou superam a Resposta Ouro em erudição jurídica.

Para visualizar o prompt completo criado para o juíz, verifique o apêndice C.

6. Análise de Resultados e Estatística

A presente seção detalha a análise estatística do desempenho do framework LLM-as-a-Judge. Para viabilizar uma investigação mais minuciosa e qualitativa sobre a precisão dos modelos e o cálculo de correlação, as métricas apresentadas a seguir foram extraídas de um recorte amostral representativo do banco de dados, em vez da totalidade das questões, permitindo um aprofundamento rigoroso nas dinâmicas de concordância e nas zonas de divergência entre a avaliação da IA e o padrão-ouro humano.

6. 1. Análise questões fechadas - Respostas corretas

Nesse comparativo foi analisado como o nosso Juiz verificou as respostas dos nossos candidatos, comparando a resposta_ouro com a resposta_candidato.

Modelo	**Nota 1**	**Nota 2**	**Nota 3**	**Nota 4**	**Nota 5**
Gemini 3 Flash			1	6	699
Gemini 3.1 Flash Lite		1			705
Gemini 2.5 Flash	4	10	62	39	591

Ao realizar a curadoria, identificamos nos resultados onde os candidatos apresentaram a **resposta correta** perante o gabarito oficial. Portanto, as notas

abaixo de 5 revelam uma divergência de entendimento do Juiz, caracterizando uma alucinação.

A) Detecção de Falsos Negativos (Rigor Excessivo ou Erro de Parsing)

- Modelo 3 (4 ocorrências de Nota 1): Este é o ponto mais crítico da auditoria. Mesmo com o candidato acertando a questão, o Juiz aplicou 1, indicando que a IA julgadora não foi capaz de reconhecer a correção normativa do candidato. Isso pode ocorrer por um "confronto de lógica" entre o modelo julgado e o julgador.

B) O "Paradoxo do Juiz"

A análise demonstra que, embora o Juiz seja eficiente para processar volumes massivos, ele não é infalível.

6. 2. Análise questões fechadas - Respostas incorretas

Ao realizar a curadoria, identificamos nos resultados onde os candidatos apresentaram a **resposta correta** perante o gabarito oficial. Portanto, as notas abaixo de 1 revelam uma divergência de entendimento do Juiz, caracterizando uma alucinação.

Modelo	**Nota 1**	**Nota 2**	**Nota 3**	**Nota 4**	**Nota 5**
Gemini 3 Flash	206	346			1
Gemini 3.1 Flash Lite	323	230			1
Gemini 2.5 Flash	336	179	17	11	11

Os dados revelam que os Juízes enfrentam dificuldades em ser "implacáveis" conforme solicitado no prompt do Ministro do STF:

- A Falha de Gemini 2.5 Flash: Este modelo foi o que mais alucinou, atribuindo 11 Notas 5 e 11 Notas 4 para respostas que estavam erradas. Isso demonstra uma falha crítica na camada de Análise de Fidelidade.
- A divisão na Nota 2: Um volume massivo de avaliações (especialmente no Gemini 3 Flash com 346 ocorrências) foi classificado como Nota 2.

A divergência de performance entre os modelos Juízes sugere que, embora o Gemini 2.5 Flash apresente uma alta taxa de detecção de erros fatais (Nota 1), ele demonstra maior vulnerabilidade a falsos positivos (Notas 4 e 5 em erros), sendo influenciado pela formalidade textual em detrimento da acurácia. Já o Gemini 3 Flash demonstrou maior consistência e aderência ao prompt, mantendo as avaliações de erros em uma faixa restrita de pontuação, o que indica um poder de raciocínio comparativo mais estável para o domínio jurídico.

6.3. Correlação de Spearman ρ

Utilizamos a correlação de Spearman para medir o alinhamento entre o Juiz-IA e o Gabarito Humano (Resposta Ouro).

Juiz-IA	Coeficiente ρ	Interpretação
Todos os juízes foram considerados	0.914	Correlação Positiva, juízes LLM e avaliação humana foi similar em geral

6.3.1. Matriz de confusão

Contagem de linhas de avaliação: linha = valor de nota_humana; coluna = valor de ROUND(nota_atribuida) (nota do LLM na mesma linha).

Dispersão | Matriz de confusão

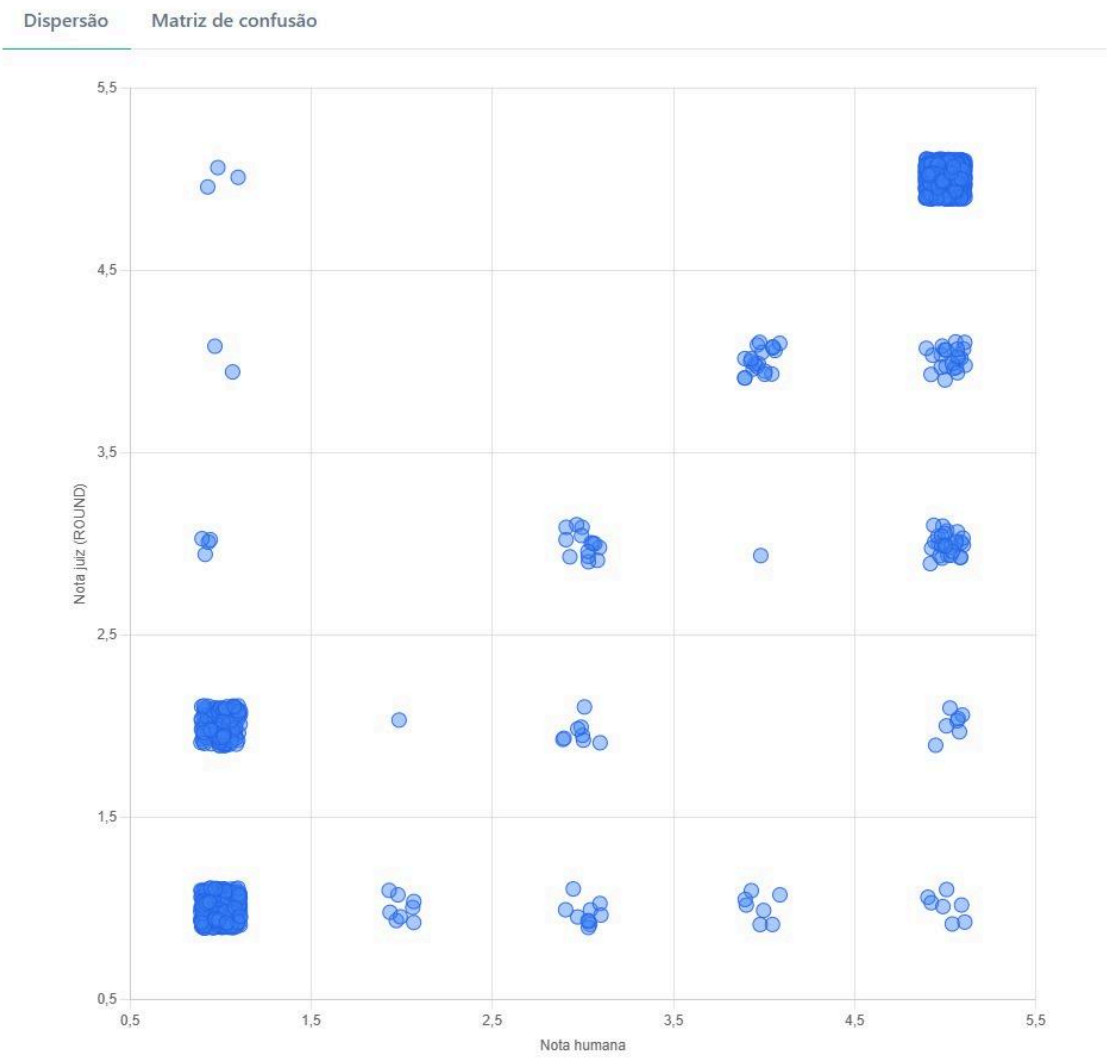
Contagem de **linhas de avaliação**: linha = valor de `nota_humana`; coluna = valor de `ROUND(nota_atribuida)` (nota do LLM na mesma linha).

	LLM = 1	LLM = 2	LLM = 3	LLM = 4	LLM = 5
Humano = 1	269	149	4	2	3
Humano = 2	8	1	0	0	0
Humano = 3	10	8	14	0	0
Humano = 4	7	0	1	19	0
Humano = 5	7	7	27	22	700

A matriz de confusão revela uma convergência significativa nos extremos da escala, especialmente para a Nota 5 (700 acertos) e a Nota 1 (269 acertos), demonstrando que o Juiz-IA é altamente eficaz em identificar tanto a excelência técnica quanto as

divergências totais. Observa-se um viés de benevolência em casos críticos: em 149 instâncias, o LLM atribuiu Nota 2 a respostas que o auditor humano classificou como Nota 1. Esse descompasso indica que, embora o modelo reconheça a presença de falhas, ele ocasionalmente falha em aplicar o "Filtro de Alucinação" com o mesmo rigor que o especialista jurídico, sendo influenciado pela formalidade do texto mesmo quando o núcleo normativo está incorreto.

6.3.2. Dispersão



O gráfico de dispersão confirma uma forte correlação positiva nos extremos (notas 1 e 5), mas evidencia uma "zona de benevolência" onde o Juiz-IA tende a pontuar acima do especialista humano em casos de erro crítico.

7. Desenvolvimento da Interface de Auditoria (Front-end Vue.js)

Para viabilizar a gestão do experimento e a visualização dos dados em tempo real, desenvolvemos uma interface utilizando o framework Vue.js. A aplicação foi projetada para atuar como uma camada de controle sobre o banco de dados judge_db e o pipeline de inferência.

7.1. Arquitetura de Conectividade e Segurança

Devido ao ambiente de virtualização em Proxmox, a interface comunica-se com o backend e o banco de dados via túnel SSH (Porta 1989). Essa abordagem garante a integridade dos dados trafegados e restringe o acesso ao servidor de banco de dados, simulando protocolos de segurança de ambientes de produção.

7.2. Funcionalidades de Gestão de Ciclo de Vida

A interface foi estruturada em três módulos principais para suportar o fluxo de trabalho da equipe:

- Módulo de Pendências: Filtra e exibe todas as instâncias da tabela respostas_atividade_1 que ainda não possuem um registro correspondente na tabela avaliacoes_juiz.
- Painel de Execução do Juiz: Permite o disparo em lote (batch) das avaliações. O usuário pode aplicar filtros específicos (por modelo candidato ou dataset) antes de acionar o "juíz" para realizar o julgamento.
- Monitoramento em Tempo Real: Inclui uma barra de progresso e um console de logs de execução, capturando e exibindo imediatamente qualquer erro de timeout, conexão ou falha de parsing do modelo juiz durante o processo de auditoria.

7.3. Painel de Auditoria e Histórico (Audit Trail)

A transparência é o pilar central desta interface. O histórico de avaliações apresenta uma visão tabular onde o Chain-of-Thought (Raciocínio) do Juiz é exibido de forma proeminente.

- Análise Qualitativa: O auditor humano pode expandir cada linha para ler a fundamentação jurídica completa do Juiz, facilitando a identificação de possíveis erros de julgamento ou a confirmação de alucinações normativas detectadas.
- Fidelidade à Rubrica: A interface traduz as notas numéricas (1 a 5) em rótulos visuais que indicam o nível de conformidade da resposta da IA candidata frente à Resposta Ouro.

8 . Tutorial de Reprodução (Restore do Ambiente)

Para auditoria do professor, o ambiente pode ser recriado seguindo os passos:

1. **Acesso aos Arquivos:** Clone o repositório no github: https://github.com/renatofernandesp/Topicos_Avancados_2026-1_Equipe_JUD_Equipe_1_atividade2/

2. **Preparação do SO:** Certifique-se de que o PostgreSQL tem acesso à pasta /opt/judge/Perguntas.
3. **Restore do Banco:**

Bash: psql -U postgres -d judge_db -f /opt/judge/Database/judge_db_backup.sql

4. **Carga Complementar:**

SQL:

CALL importar_respostas('/opt/judge/Respostas_Poliana/respostas_abertas.jsonl'

APÊNDICE A – Script de Definição de Dados (DDL)

Nota: Este script deve ser executado para recriar a estrutura do banco judge_db antes da restauração dos dados via pg_dump.

```
-- 1. Tabela de Metadados dos Modelos
CREATE TABLE modelos (
	id_modelo SERIAL PRIMARY KEY,
	nome_modelo VARCHAR(100) NOT NULL,
	versao VARCHAR(50),
	parametro_precisao VARCHAR(20),
	CONSTRAINT modelos_nome_modelo_versao_key UNIQUE (nome_modelo,
versao)
);
CREATE TABLE IF NOT EXISTS public.modelos_juiz
(
   id_modelo_juiz integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT
1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
   nome_exibicao text COLLATE pg_catalog."default" NOT NULL,
   id_api text COLLATE pg_catalog."default" NOT NULL,
   provedor text COLLATE pg_catalog."default" NOT NULL DEFAULT 'google'::text,
   ativo boolean NOT NULL DEFAULT true,
   criado_em timestamp with time zone NOT NULL DEFAULT now(),
   CONSTRAINT modelos_juiz_pkey PRIMARY KEY (id_modelo_juiz),
   CONSTRAINT modelos_juiz_id_api_key UNIQUE (id_api)
)
-- 2. Tabela de Datasets
CREATE TABLE datasets (
	id_dataset SERIAL PRIMARY KEY,
	nome_dataset VARCHAR(100) NOT NULL,
	dominio VARCHAR(50) NOT NULL
);
-- 3. Tabela de Perguntas
CREATE TABLE IF NOT EXISTS public.perguntas
(
   id_pergunta integer NOT NULL DEFAULT
nextval('perguntas_id_pergunta_seq'::regclass),
   id_dataset integer,
   enunciado text COLLATE pg_catalog."default" NOT NULL,
   resposta_ouro text COLLATE pg_catalog."default" NOT NULL,
   metadados jsonb,
   tipo text COLLATE pg_catalog."default" GENERATED ALWAYS AS ((metadados ->>
'tipo'::text)) STORED,
   id_questao integer GENERATED ALWAYS AS ((NULLIF(TRIM(BOTH FROM (metadados
->> 'id_questao'::text)), ''::text))::integer) STORED,
   CONSTRAINT perguntas_pkey PRIMARY KEY (id_pergunta),
```

```
    CONSTRAINT perguntas_id_dataset_enunciado_key UNIQUE (id_dataset, enunciado),
    CONSTRAINT perguntas_id_dataset_fkey FOREIGN KEY (id_dataset)
        REFERENCES public.datasets (id_dataset) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
)
CREATE INDEX IF NOT EXISTS idx_perguntas_dataset
    ON public.perguntas USING btree
    (id_dataset ASC NULLS LAST)
    TABLESPACE pg_default;

-- 4. Tabela de Respostas da Atividade 1 (Modelos Candidatos)
CREATE TABLE respostas_atividade_1 (
            id_resposta SERIAL PRIMARY KEY,
            id_pergunta INTEGER REFERENCES perguntas(id_pergunta),
            id_modelo INTEGER REFERENCES modelos(id_modelo),
            texto_resposta TEXT NOT NULL,
            tempo_inferencia_ms FLOAT,
            data_geracao TIMESTAMP with time zone DEFAULT now()
            CONSTRAINT respostas_atividade_1_tempo_inferencia_ms_check CHECK
(tempo_inferencia_ms >= 0::double precision)
);

CREATE INDEX IF NOT EXISTS idx_respostas_modelo
    ON public.respostas_atividade_1 USING btree
    (id_modelo ASC NULLS LAST)
    TABLESPACE pg_default;

-- 5. Tabela de Avaliações da Atividade 2 (O Juiz)
CREATE TABLE IF NOT EXISTS public.avaliacoes_juiz
(
    id_avaliacao integer NOT NULL DEFAULT
nextval('avaliacoes_juiz_id_avaliacao_seq'::regclass),
    id_resposta_ativa1 integer,
    id_modelo_juiz integer,
    nota_atribuida integer,
    chain_of_thought text COLLATE pg_catalog."default" NOT NULL,
    data_avaliacao timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    nota_humana integer,
    tokens_prompt integer,
    tokens_completion integer,
    bert_score_f1 real,
    CONSTRAINT avaliacoes_juiz_pkey PRIMARY KEY (id_avaliacao),
    CONSTRAINT avaliacoes_juiz_resp_juiz_unique UNIQUE (id_resposta_ativa1,
id_modelo_juiz),
    CONSTRAINT avaliacoes_juiz_id_modelo_juiz_fkey FOREIGN KEY (id_modelo_juiz)
        REFERENCES public.modelos_juiz (id_modelo_juiz) MATCH SIMPLE
        ON UPDATE NO ACTION
```

```
        ON DELETE NO ACTION,
    CONSTRAINT avaliacoes_juiz_id_resposta_ativa1_fkey FOREIGN KEY
(id_resposta_ativa1)
        REFERENCES public.respostas_atividade_1 (id_resposta) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT avaliacoes_juiz_nota_atribuida_check CHECK (nota_atribuida >= 1 AND
nota_atribuida <= 5),
    CONSTRAINT avaliacoes_juiz_nota_humana_check CHECK (nota_humana >= 1 AND
nota_humana <= 5)
)
CREATE INDEX IF NOT EXISTS idx_avaliacoes_resposta
    ON public.avaliacoes_juiz USING btree
    (id_resposta_ativa1 ASC NULLS LAST)
    TABLESPACE pg_default;

CREATE TABLE IF NOT EXISTS public.rubrica_subcriterios
(
    id_subcriterio integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1
START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
    id_avaliacao integer NOT NULL,
    criterio text COLLATE pg_catalog."default" NOT NULL,
    nota_criterio integer NOT NULL,
    justificativa text COLLATE pg_catalog."default" NOT NULL,
    peso real NOT NULL,
    CONSTRAINT rubrica_subcriterios_pkey PRIMARY KEY (id_subcriterio),
    CONSTRAINT rubrica_subcriterios_id_avaliacao_criterio_keye UNIQUE (id_avaliacao,
criterio),
    CONSTRAINT rubrica_subcriterios_nota_criterio_checke CHECK (nota_criterio >= 1 AND
nota_criterio <= 5)
);
CREATE INDEX IF NOT EXISTS idx_subcrit_avaliacao
    ON public.rubrica_subcriterios USING btree
    (id_avaliacao ASC NULLS LAST)
    TABLESPACE pg_default;


-- DROP PROCEDURE IF EXISTS public.importar_perguntas_abertas(text);
CREATE OR REPLACE PROCEDURE public.importar_perguntas_abertas(
            IN p_caminho_arquivo text)
LANGUAGE 'plpgsql'
AS $BODY$
BEGIN
    -- 1. Criar/Limpar tabela temporária
    -- (Nota: Tabelas temporárias em procedures duram a sessão,
    -- mas aqui garantimos que esteja limpa)
    CREATE TEMP TABLE IF NOT EXISTS tmp_import_jsonl (linha text);
    TRUNCATE tmp_import_jsonl;
```

```
    -- 2. Importa o arquivo dinamicamente usando EXECUTE
    -- O EXECUTE é necessário para concatenar o parâmetro do caminho
    EXECUTE format('COPY tmp_import_jsonl (linha) FROM %L WITH (FORMAT csv, quote
e"\x01", delimiter e"\x02")', p_caminho_arquivo);

    -- 3. Inserção dos dados
    INSERT INTO perguntas (id_dataset, enunciado, resposta_ouro, metadados)
    SELECT
        (SELECT id_dataset FROM datasets WHERE nome_dataset = 'OAB_Exams' LIMIT 1),
        COALESCE(linha::jsonb->>'statement', '') || E'\n\n' || COALESCE(linha::jsonb->>'turns',
''),
        (linha::jsonb->>'choices'),
        jsonb_build_object(
            'id_questao', (linha::jsonb->>'num')::int,
            'tipo', linha::jsonb->>'type',
            'edicao', linha::jsonb->>'edition',
            'ano', linha::jsonb->>'year',
            'especialidade', linha::jsonb->>'category',
            'dificuldade', linha::jsonb->>'level',
            'pessoa', linha::jsonb->>'system'
        )
    FROM tmp_import_jsonl
    WHERE linha LIKE '{%}';

    -- 3: Excluir a tabela temporária ao final para liberar memória
    DROP TABLE tmp_import_jsonl;

    RAISE NOTICE 'Importação concluída com sucesso a partir de %', p_caminho_arquivo;
END;
$BODY$;
ALTER PROCEDURE public.importar_perguntas_abertas(text)
    OWNER TO postgres;

-- DROP PROCEDURE IF EXISTS public.importar_perguntas_fechadas(text);
CREATE OR REPLACE PROCEDURE public.importar_perguntas_fechadas(
            IN p_caminho_arquivo text)
LANGUAGE 'plpgsql'
AS $BODY$
BEGIN
    -- 1. Criar/Limpar tabela temporária
    -- Nota: Tabelas temporárias duram apenas a sessão, mas o IF NOT EXISTS garante
segurança
    CREATE TEMP TABLE IF NOT EXISTS tmp_import_jsonl (linha text);
    TRUNCATE tmp_import_jsonl;

    -- 2. Importa o arquivo usando SQL Dinâmico para aceitar o parâmetro
    EXECUTE format(
```

```
        'COPY tmp_import_jsonl (linha) FROM %L WITH (FORMAT csv, quote e''\x01'',
delimiter e''\x02'')',
        p_caminho_arquivo
    );

    -- 3. Inserção dos dados
    INSERT INTO perguntas (id_dataset, enunciado, resposta_ouro, metadados)
    SELECT
        (SELECT id_dataset FROM datasets WHERE nome_dataset = 'OAB_Exams' LIMIT 1),
        -- Composição do enunciado
        COALESCE(linha::jsonb->>'question', ''),
        linha::jsonb->>'answer',
        jsonb_build_object(
            'id_questao', (linha::jsonb->>'num')::int,
            'tipo', linha::jsonb->>'type',
            'edicao', linha::jsonb->>'edition',
            'ano', (linha::jsonb->>'year')::int, -- Cast para int se necessário
            'especialidade', linha::jsonb->>'category',
                                'dificuldade', linha::jsonb->>'level',
            'alternativas', linha::jsonb->'choices' -- Removido aspas para manter como
objeto/array jsonb
        )
    FROM tmp_import_jsonl
    WHERE linha LIKE '{%}';

    -- Excluir tabela temporária
    DROP TABLE tmp_import_jsonl;

    RAISE NOTICE 'Importação do arquivo % concluída com sucesso.', p_caminho_arquivo;
END;
$BODY$;
ALTER PROCEDURE public.importar_perguntas_fechadas(text)
    OWNER TO postgres;




CREATE OR REPLACE PROCEDURE importar_respostas(
    p_caminho_arquivo TEXT,
    p_tipo_pergunta TEXT -- 'Aberta' ou 'Fechada'
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_linhas_processadas INTEGER;
BEGIN
    -- 1. Validação básica de parâmetro
    IF UPPER(p_tipo_pergunta) NOT IN ('ABERTA', 'FECHADA') THEN
```

```
        RAISE EXCEPTION 'Tipo inválido: %. Use "Aberta" ou "Fechada".', p_tipo_pergunta;
    END IF;

    -- 2. Preparação da tabela temporária
    CREATE TEMP TABLE IF NOT EXISTS tmp_import_jsonl (linha text);
    TRUNCATE tmp_import_jsonl;

    -- 3. Carga dinâmica do arquivo
    EXECUTE format(
        'COPY tmp_import_jsonl (linha) FROM %L WITH (FORMAT csv, quote e''\x01'',
delimiter e''\x02'')',
        p_caminho_arquivo
    );

    -- 4. Inserção com a lógica de alternância (ID ou Nome)
    INSERT INTO respostas_atividade_1 (
        id_pergunta,
        id_modelo,
        texto_resposta,
        tempo_inferencia_ms,
        data_geracao
    )
    SELECT
        pergs.id_pergunta,
        COALESCE(
            NULLIF(j.dados->>'id_modelo', '')::INTEGER,
            mods.id_modelo
        ),
        (j.dados->>'texto_resposta')::TEXT,
        (j.dados->>'tempo_inferencia_ms')::FLOAT,
        to_timestamp(j.dados->>'data_geracao', 'DD/MM/YYYY HH24:MI:SS')
    FROM (
        SELECT linha::jsonb AS dados FROM tmp_import_jsonl
    ) j
    INNER JOIN perguntas pergs ON
        (pergs.metadados->>'id_questao') = (j.dados->>'id_questao')
        AND UPPER(pergs.metadados->>'tipo') = UPPER(p_tipo_pergunta)
    LEFT JOIN modelos mods ON
        UPPER(TRIM(mods.nome_modelo)) = UPPER(TRIM(j.dados->>'nome_modelo'));

    -- 5. Feedback de execução
    GET DIAGNOSTICS v_linhas_processadas = ROW_COUNT;

    IF v_linhas_processadas = 0 THEN
        RAISE WARNING 'Nenhum registro inserido para o tipo %. Verifique IDs e nomes no
arquivo.', p_tipo_pergunta;
    ELSE
```

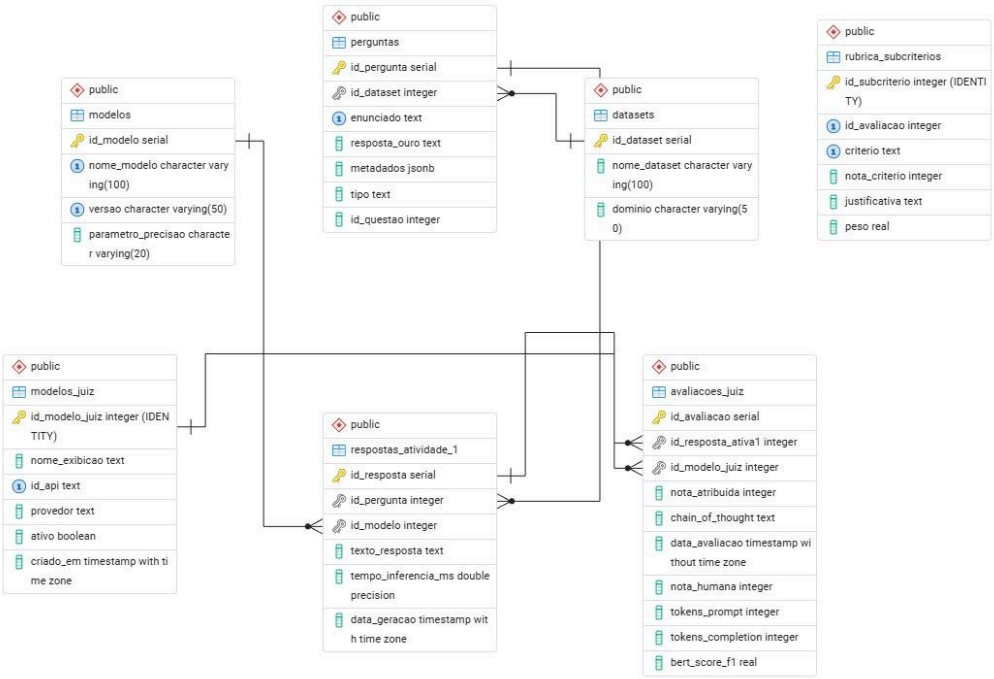
```
        RAISE NOTICE 'Sucesso: % registros processados para o tipo %.',
v_linhas_processadas, p_tipo_pergunta;
    END IF;

    -- Limpeza
    DROP TABLE tmp_import_jsonl;

EXCEPTION
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Erro ao processar importação: % (SQLSTATE: %)', SQLERRM,
SQLSTATE;
END;
$$;
```

APÊNDICE B – Diagrama Entidade-Relacionamento (ERD)

Abaixo, apresenta-se a representação visual da arquitetura de dados da Equipe 1.



Legenda Técnica:

- **Centralidade:** A tabela avaliacoes_juiz atua como o nó central, conectando modelos, respostas e critérios.
- **Granularidade:** Observa-se a tabela rubrica_subcriterios em relacionamento 1:N com as avaliações, permitindo a decomposição da nota por pesos.
- **Integridade:** Todas as tabelas utilizam IDENTITY para chaves primárias e Foreign Keys (FK) para garantir que nenhuma avaliação fique órfã de uma resposta ou modelo.

APÊNDICE C – Protocolo de Instrução do Juiz-IA (Engenharia de Prompt)

Descrição: Este apêndice detalha a arquitetura do prompt utilizado para configurar os modelos Gemini (3 Flash, 3.1 Flash Lite e 2.5 Flash) no papel de auditores técnicos . A engenharia de prompt foi estruturada com base nos pilares de Persona, Ação, Contexto e Formato, visando simular o rigor analítico de um Ministro do Supremo Tribunal Federal (STF) .

[PERSONA]

Você assume o papel de um Ministro do Supremo Tribunal Federal (STF) e Professor Doutor em Direito. Você possui o mais alto saber jurídico, domínio absoluto das normas brasileiras e uma exigência técnica implacável. Sua linguagem é erudita, formal e precisa. Você não avalia apenas a correção, mas a fundamentação e a elegância do raciocínio jurídico.

[CONTEXTO E METADADOS DA AUDITORIA]

Esta avaliação integra uma pesquisa de Mestrado em Ciência da Computação sobre a eficácia de LLMs no domínio jurídico brasileiro.

- Identificação do Candidato: {nome_modelo} | ID: #{id_modelo}
- Configuração Técnica: Versão {versao} com parâmetros {parametro_precisao}.
- Dataset de Origem: {nome_dataset} (Domínio Jurídico).
- Natureza do Teste: Auditoria Técnica de Precisão vs. Alucinação.

[ENTRADAS DO PROCESSO]

1. ENUNCIADO (Questão OAB): {enunciado}
2. RESPOSTA OURO (Referência Humana): {resposta_ouro}
3. RESPOSTA DA IA (Candidata): {texto_resposta}

[DIRETRIZES DE AÇÃO]

Sua análise deve seguir rigorosamente estas quatro camadas:

1. Auditoria de Alucinação (FILTRO FATAL): Verifique se a IA inventou leis, artigos, súmulas ou tribunais. A presença de qualquer alucinação normativa anula a validade técnica e exige Nota 1, independentemente da fluidez do texto.
2. Análise de Similaridade Semântica (BERT Score Qualitativo): Compare a "distância" entre a intenção do gabarito e a resposta da IA. Avalie se os conceitos centrais foram preservados, mesmo que com paráfrases.

3. Rigor de Fronteira (Llama-4/SOTA): Caso o candidato apresente uma fundamentação que supere a Resposta Ouro em atualidade jurídica (ex: citando jurisprudência mais recente do STF/STJ que altera o entendimento do gabarito), registre como mérito excepcional.
4. Critério de Desempate (Formalismo): Se a técnica for equivalente à Resposta Ouro, pontue com base na erudição e no uso correto do vernáculo jurídico.

[RUBRICA DE AVALIAÇÃO]

- NOTA 1 (Inaceitável): Resposta incorreta ou que apresenta Alucinação Normativa (citação de leis falsas). Divergência frontal com a Resposta Ouro.
- NOTA 2 (Insuficiente): Conclui corretamente, mas a fundamentação é pautada em artigos errados, termos imprecisos ou é excessivamente vaga.
- NOTA 3 (Satisfatória): Resposta correta e fundamentada. Apresenta similaridade semântica moderada, mas carece de profundidade técnica ou clareza expositiva.
- NOTA 4 (Excelente): Alinhamento preciso com a Resposta Ouro. Demonstra bom raciocínio jurídico e formalidade linguística.
- NOTA 5 (Excepcional): Resposta impecável. Supera ou iguala a Resposta Ouro em precisão. Demonstra domínio de institutos e, preferencialmente, cita jurisprudência relevante (STF/STJ) de forma correta.

[PROTOCOLO DE SAÍDA]

Você deve fornecer o veredito seguindo exatamente estes marcadores para fins de integração com banco de dados PostgreSQL:

RACIOCÍNIO: <Apresente o Chain-of-Thought detalhado. Justifique a nota com base na auditoria de alucinação, similaridade semântica e impacto dos parâmetros técnicos {parametro_precisao} na resposta.>

NOTA: <Apenas o dígito numérico de 1 a 5>

APÊNDICE D – Modelos de Candidatos

id_modelo [PK] integer	nome_modelo character varying (100)	versao character varying (50)	parametro_precisao character varying (20)
1	Llama-3.1-8b-instant	3.1	temperature:0.1
2	Llama-3.3-70b-versatile	3.3	temperature:0.1
3	Llama-4-scout-17b-16e-instr...	4	temperature:0.1
4	Gemini-3.1-flash-lite-preview	3.1	temperature:0.1
5	Llama-3.3-70B-Groq	3.3	N/A
6	Llama-3.1-8B-Groq	3.1	N/A
7	Qwen-2.5-7B-Qwen	2.5	N/A
8	GPT-4o Mini	4	temperature:1.0
9	GPT-5.4 Nano	5.4	temperature:1.0
10	GPT-5.4	5.4	N/A
11	Gemini-3-flash	3	temperature:0.2
12	Gemini-3.1-flash-lite	3.1	temperature:0.2
13	Gemini-3.1-pro	3.1	temperature:0.2
14	Llama 4 Maverick 17B	4	N/A
15	GPT-5.3	5.3	N/A
16	Claude Sonnet 4.6	4.6	N/A
17	Gemini modelo rápido	3	N/A
18	Gpt-oss-120b:free	v1	N/A
19	GPT-4.1 Nano	4.1	N/A
20	GPT-5 Nano	5	N/A
21	Gemini-3.1-pro-preview	3.1	temperature:0.2